

Branching Strategy — Правила работы с ветками

Как создавать, именовать и вести ветки в проекте TimeToEvent.

Зачем нужен стандарт?

- **Понятная история** — по имени ветки сразу видно, что в ней делается
- **Автоматизация** — CI/CD может реагировать на типы веток
- **Чистый main** — в main попадает только проверенный код
- **Параллельная работа** — несколько фич можно делать одновременно
- **Связь с коммитами** — ветка и её коммиты говорят об одном и том же

Формат имени ветки

`<type>/<optional-scope>-<description>`

Правила

1. **Только lowercase** — никаких CamelCase или PascalCase
2. **Дефис вместо пробела** — `use-timer-hook`, не `use timer hook`
3. **Слэш после типа** — `feature/`, `fix/`, `docs/`
4. **Краткое описание** — 2-4 слова, глагол в инфинитиве
5. **Без ID issue** (пока нет трекера) — добавим позже, когда заведём GitHub Issues

Типы веток

feature/ — новая фича

Основной тип для разработки функциональности.

Пример	Описание
feature/color-picker	Компонент выбора цвета
feature/datetime-picker	Компонент выбора даты и времени
feature/live-timers	Живые таймеры countdown/countup
feature/pairing-ui	UI для сопряжения устройств
feature/sync-status	Индикатор статуса синхронизации

fix/ — исправление бага

Только для исправления существующего функционала.

Пример	Описание
fix/timer-freeze	Таймер не обновляется в реальном времени
fix/mdns-discovery	mDNS не находит устройства в подсети
fix/ws-reconnect	WebSocket не переподключается после разрыва
fix/event-date-parse	Неправильный парсинг даты в форме

docs/ — документация

Изменения в .md файлах, комментарии, JSDoc/TSDoc.

Пример	Описание
docs/commands-reference	Справочник команд (COMMANDS.md)
docs/conventional-commits	Гайд по коммитам (COMMITTS.md)
docs/install-update	Обновление INSTALL.md

Пример	Описание
docs/architecture-diagram	Добавить диаграмму в ARCHITECTURE.md

refactor/ — рефакторинг

Изменение структуры кода без изменения поведения.

Пример	Описание
refactor/extract-date-utils	Вынести утилиты дат в отдельный модуль
refactor/events-store	Упростить Zustand store
refactor/ws-server	Разделить WsServer на классы

test/ — тесты

Добавление или изменение тестов.

Пример	Описание
test/crypto-unit	Unit-тесты для crypto модуля
test/events-store	Тесты Zustand store
test/e2e-pairing	E2E тест сопряжения

chore/ — рутина

Зависимости, конфиги, CI, скрипты — всё, что не влияет на код.

Пример	Описание
chore/update-tauri	Обновить Tauri до 2.1.x
chore/github-actions	Настроить CI в GitHub Actions
chore/husky-hooks	Добавить pre-commit хуки
chore/remove-dead-code	Удалить неиспользуемые файлы

style/ — форматирование

Только косметические изменения (пробелы, точки с запятой).

Пример	Описание
style/fix-indentation	Исправить отступы в events.rs
style/prettier-run	Прогнать Prettier по всему проекту

perf/ — производительность

Оптимизации без изменения функциональности.

Пример	Описание
perf/virtualize-lists	Виртуализация списков событий
perf/sqlite-indexes	Добавить индексы для частых запросов

release/ — подготовка релиза

Ветки для подготовки конкретного релиза.

Пример	Описание
release/v0.2.0	Подготовка релиза 0.2.0
release/v1.0.0	Подготовка первого публичного релиза

hotfix/ — срочное исправление

Критичный баг в production, чинится напрямую от тега релиза.

Пример	Описание
hotfix/crash-on-start	Приложение падает при запуске
hotfix/data-loss	Потеря данных при синхронизации

Workflow создания ветки

1. Убедись, что ты на актуальном main

```
git checkout main
git pull origin main
```

2. Создай ветку

```
git checkout -b feature/color-picker
```

Или одной командой:

```
git checkout -b feature/color-picker main
```

3. Делай коммиты по Conventional Commits

```
git commit -m "feat(ui): добавить ColorPicker компонент"
git commit -m "feat(ui): добавить HEX-ввод в ColorPicker"
git commit -m "test(ui): добавить тесты ColorPicker"
```

4. Регулярно подтягивай main

```
# Пока работаешь в ветке
git fetch origin
git rebase origin/main
```

Почему rebase, а не merge? — чтобы история оставалась линейной и понятной.

5. Запушь ветку

```
git push -u origin feature/color-picker
```

Флаг `-u` запоминает remote, дальше можно просто `git push`.

6. Создай Pull Request

На GitHub:

1. Открой ветку → кнопка **"Compare & pull request"**
2. Заполни шаблон PR (см. [CONTRIBUTING.md](#))
3. Назначь ревьюера (или подожди 24 часа для solo-проекта)

7. Merge и удаление

После merge:

```
git checkout main
git pull origin main
git branch -d feature/color-picker      # локально
git push origin --delete feature/color-picker # на remote
```

Что НЕ делать

✗ Не работай напрямую в main

```
# ПЛОХО
git checkout main
# ... правишь код ...
git commit -m "stuff"
git push
```

✗ Не используй общие имена

```
# ПЛОХО
git checkout -b my-branch
git checkout -b fix
git checkout -b test
git checkout -b new-feature
```

✗ Не используй camelCase или пробелы

ПЛОХО

```
git checkout -b feature/ColorPicker
git checkout -b feature/color picker
git checkout -b feature/colorPicker
```

✗ Не держи ветки вечно

Ветка живёт **пока задача не сделана**. Если задача большая — разбей на подзадачи и создай несколько веток.

✗ Не merge'ай ветку в себя

ПЛОХО — ты в feature/color-picker и делаешь:

```
git merge feature/color-picker
```



Когда какой тип использовать

Ситуация	Тип ветки
Добавляешь новую фичу	feature/
Чинишь баг	fix/
Пишешь/правишь документацию	docs/
Меняешь структуру без изменения поведения	refactor/
Добавляешь тесты	test/
Обновляешь зависимости, конфиги, CI	chore/
Только форматирование	style/
Оптимизируешь производительность	perf/
Готовишь релиз	release/
Срочно чинишь production	hotfix/

Связь с Conventional Commits

Имя ветки и коммиты внутри неё должны **согласовываться**:

Ветка: `feature/color-picker`

Коммиты: `feat(ui): добавить ColorPicker`

`feat(ui): добавить HEX-ввод`

`test(ui): добавить тесты`

Ветка: `fix/timer-freeze`

Коммиты: `fix(timers): исправить обновление каждую секунду`

`test(timers): добавить тест useTimer`

Ветка: `docs/commands-reference`

Коммиты: `docs: добавить COMMANDS.md`

`docs: обновить ссылки в INSTALL.md`

Правило: тип ветки = тип коммитов внутри неё. Не должно быть `fix/` ветки с `feat:` коммитами.

Связанные документы

- `COMMITTS.md` — правила написания коммитов
- `CONTRIBUTING.md` — гайд для контрибьюторов
- `TODO` — бэклог задач (отсюда берём задачи для веток)